S0 · ML E IA: DE DESARROLLO A PRODUCCIÓN

# Tu equipo listo para S1

Instalar, crear el entorno y comprobarlo: todo para copiar y pegar, en el orden correcto y con una comprobación al final de cada paso.

**Windows 10 (1809+) o 11** **Entorno env-ia-mlops** **Caso: Bank Marketing**

### CÓMO USAR ESTAS DIAPOSITIVAS

Copia cada línea y pégala en la terminal que indica la diapositiva. No pases al siguiente paso si lo que ves no coincide con «Debe salir».

Los comandos de Python, conda, pytest, SQLite y MLflow se ejecutaron el 09/11/2026 en Linux, en un entorno recién creado con el `environment.yml` actual; las salidas que ves son las reales. **Nada se probó en un Windows real**: cada diapositiva lo indica en su etiqueta.

EL ORDEN LÓGICO

# Seis bloques, siempre en este orden

**A** → **Programas**
Git · VS Code · Miniconda
**CMD o PowerShell**

**B** → **Conda**
solo conda-forge
**Anaconda Prompt**

**C** → **Repositorio**
clonar la rama develop
**CMD o PowerShell**

**D** → **Entorno**
crear · activar · humo
**Anaconda Prompt**

**E** → **Pruebas para S1**
VS Code · pytest · SQLite · MLflow · dato
**Anaconda Prompt + VS Code**

**F** → **Nube**
no bloquea S1
**según el paso**

## POR QUÉ ESTE ORDEN

- **Git antes de clonar**, y **Miniconda antes de crear el entorno**.
- **Los canales de conda antes de crear**: si no, conda puede negarse por los términos de servicio.
- **El repositorio antes que el entorno**: la receta ( `environment.yml` ) está dentro.
- **El entorno antes de las pruebas**: todas usan su Python.

## LA REGLA

Cada paso termina con una comprobación (**Debe salir**). Si no sale igual, **para**: lee «Si falla». Seguir con un paso roto solo produce errores más raros después.

El bloque F (nube) no bloquea S1: puede esperar.

ANTES DE EMPEZAR

# Lo que necesitas y qué terminal usar

## REQUISITOS

- Windows 10 (1809 o superior) o Windows 11. Esta edición no tiene guía para macOS ni Linux.
- Reserva unos **6 GB libres** (estimación): el entorno ocupa unos 2,5 GB (medido en Linux) y hay que sumar la caché de descargas de conda y los programas.
- Conexión estable: se descargan cientos de MB.
- Una cuenta de GitHub.
- No hace falta ser administrador: winget pedirá permiso cuando haga falta.
- Si tu equipo no te deja instalar programas, usa Codespaces (diapositiva «Si algo falla y cómo pedir ayuda»).

## QUÉ TERMINAL PARA QUÉ

| TERMINAL | PARA |
|---|---|
| **CMD** o **PowerShell** | winget, Git, VS Code, clonar (bloques A y C) |
| **Anaconda Prompt** (Miniconda3) | conda y Python (B, D y E) |
| Terminal de **VS Code** | lo mismo que Anaconda Prompt, si el entorno está activo |

Sabrás que el entorno está activo porque el prompt empieza por **(env-ia-mlops)**.

## DOS HÁBITOS QUE EVITAN LA MITAD DE LOS ERRORES

- Tras instalar un programa, **cierra y abre la terminal**.
- Copia cada línea y pégala con **Ctrl+V**: no escribas los comandos a mano.

 SIN PROBAR EN WINDOWS

# Comprueba winget

En: **CMD o PowerShell**

winget es el instalador de Windows que usan los comandos de este bloque. Viene con Windows 10 (1809 o superior) y Windows 11.

TERMINAL

```
> winget --version
```

Cópialas una a una.

### DEBE SALIR

Un número de versión, por ejemplo `v1.x`.

### SI FALLA

- Si no se reconoce, instala «Instalador de aplicación» desde Microsoft Store, o usa el enlace de descarga manual de cada programa.
- Si tu equipo es corporativo y no te deja instalar programas, ve a «Si algo falla y cómo pedir ayuda» (al final): hay una ruta sin instalación (Codespaces).

PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# Git y tu carpeta de trabajo

En: **CMD o PowerShell**

Git descarga el material del curso y guarda tus cambios. Cierra y vuelve a abrir la terminal después de instalar: si no, no lo reconoce.

TERMINAL

```
> winget install --id Git.Git -e --accept-source-agreements --accept-package-agreements
```

```
> git --version
```
en una terminal NUEVA

```
> git config --global user.name "Tu Nombre"
```
cambia los datos

```
> git config --global user.email "tu@correo.com"
```
cambia los datos

```
> git config --global core.longpaths true
```

```
> mkdir C:\cursos
```
una ruta corta, fuera de OneDrive y sin espacios

Cópialas una a una: algunas van en otra terminal o llevan tus datos.

## DEBE SALIR

`git version 2.x…` Los demás comandos no imprimen nada si salen bien.

```
git version 2.47.3
```

Salida real en Linux; en Windows dirá algo como `2.x.windows.1`.

## SI FALLA

- `git` no se reconoce: cierra y abre la terminal. Si sigue igual, descárgalo de git-scm.com/download/win.
- Las rutas largas (OneDrive, carpetas con espacios) dan errores en Windows: clona siempre en `C:\cursos`.

 SIN PROBAR EN WINDOWS

# Visual Studio Code y sus extensiones

En: **CMD o PowerShell**

VS Code es el editor del curso: ejecuta scripts, cuadernos y terminales. Las 7 extensiones dan Python, Jupyter, contenedores y Codespaces.

TERMINAL

```
> winget install --id Microsoft.VisualStudioCode -e --accept-source-agreements --accept-package-agreements
```

```
> code --version
```

en una terminal NUEVA

```
> code --install-extension ms-python.python
> code --install-extension ms-python.vscode-pylance
> code --install-extension ms-toolsai.jupyter
> code --install-extension ms-azuretools.vscode-containers
> code --install-extension ms-kubernetes-tools.vscode-kubernetes-tools
> code --install-extension ms-vscode-remote.remote-containers
> code --install-extension GitHub.codespaces
> code --list-extensions
```

Cópialas una a una: algunas van en otra terminal o llevan tus datos.

### DEBE SALIR

Un número de versión y un código; la lista final muestra las 7 extensiones.

### SI FALLA

- `code` no se reconoce: abre una terminal nueva o reinstala marcando «Agregar a PATH».
- Un identificador de extensión puede haber cambiado: búscala por nombre en el panel Extensiones (Ctrl+Shift+X).

PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# Miniconda

En: **Anaconda Prompt (Miniconda3)**

Miniconda instala `conda`, que crea entornos de Python aislados con las mismas versiones para todo el grupo. Se usa desde «Anaconda Prompt», no desde CMD.

TERMINAL

```
> winget install --id Anaconda.Miniconda3 -e --accept-source-agreements --accept-package-agreements
```

en CMD o PowerShell

```
> conda --version
```

ahora en «Anaconda Prompt (Miniconda3)», desde el menú Inicio

```
> conda env list
```

Cópialas una a una: algunas van en otra terminal o llevan tus datos.

## DEBE SALIR

Una versión de conda y una lista con al menos el entorno `base`. **No instales nada en `base`.**

```
conda 26.7.3
```

Salida real en Linux; tu versión puede ser otra.

## SI FALLA

- `conda` no se reconoce: no estás en «Anaconda Prompt (Miniconda3)». Búscalo en el menú Inicio.
- Si winget no encuentra el paquete, descarga Miniconda desde anaconda.com/docs/getting-started/miniconda/main.

 PROBADO EN LINUX

# Conda solo con conda-forge

En: **Anaconda Prompt (Miniconda3)**

El Miniconda de Anaconda se niega a crear entornos hasta aceptar los términos de sus canales por defecto, aunque el entorno use solo conda-forge. Esta configuración deja conda-forge como único canal y evita el bloqueo.

TERMINAL

```
> conda config --add channels conda-forge
> conda config --remove channels defaults
si dice «CondaKeyError ... not present», no pasa nada
> conda config --set channel_priority strict
> conda config --add default_channels https://conda.anaconda.org/conda-forge
> conda config --show channels default_channels
```

### DEBE SALIR

Los cuatro primeros no imprimen nada. El último debe mostrar **solo conda-forge** en `default_channels`.

### SI FALLA

- Si no hiciste esto y conda dice `CondaToSNonInteractiveError`, es este paso.
- Si tu organización necesita licencia de Anaconda, consúltalo antes: otras opciones son aceptar los términos con `conda tos accept` o usar Miniforge.

Receta probada en Linux con conda 26.9.4 (VALIDACION.md); el aviso de términos de servicio solo lo da el Miniconda de Anaconda.

 PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# Clona el curso y abre la rama develop

En: **CMD o PowerShell**

Hasta que `develop` se fusione en `main`, el material está en `develop`. Un `git clone` a secas te deja en `main`, que está casi vacío (comprobado con `git ls-remote` el 09/11/2026).

TERMINAL

```
> cd C:\cursos
> git clone --branch develop https://github.com/helicr/curso-ml-ia-desarrollo-produccion.git
> cd curso-ml-ia-desarrollo-produccion
> git branch --show-current
> code .
```

SI YA CLONASTE Y ESTÁS EN MAIN

```
> git fetch origin
> git switch develop
```

PARA TRAER NOVEDADES MÁS ADELANTE

```
> git pull
```

## DEBE SALIR

`git branch --show-current` imprime **develop** y VS Code abre la carpeta con `01-Sesiones`, `03-Entorno`, `04-Proyecto-Base` …

## SI FALLA

- Revisa tu conexión y que copiaste la URL entera. Si GitHub te pide iniciar sesión, hazlo con tu cuenta; si dice que no encuentra el repositorio, pide acceso al instructor.
- Si ves pocas carpetas, estás en `main`: usa la sección «Si ya clonaste».

`git clone --branch develop` probado contra GitHub el 09/11/2026 (clon superficial; el repositorio es público y su rama por defecto es main); `switch` probado con un repositorio de prueba. `code .`, en Windows, no.

# ¿Qué archivo de requisitos uso?

Hay tres listas y las tres fijan las **mismas versiones** (09/11/2026). Una prueba automática lo vigila. Para tu equipo, solo importa la primera.

| ARCHIVO (EN `03-ENTORNO/CONDA/`) | QUÉ ES | QUIÉN LO USA | COMANDO |
|---|---|---|---|
| `environment.yml` | Receta de conda: Python 3.11 y **23 paquetes** con versión fijada, solo de conda-forge. | **Tu equipo** (obligatorio) | `conda env create -f …` |
| `requirements-dev.txt` | Los mismos 23 paquetes, para pip. | Codespaces y GitHub Actions; alternativa sin conda | `pip install -r …` |
| `requirements.txt` | Solo lo mínimo para ejecutar la API y el modelo (7 paquetes). | La imagen Docker de S2 | `pip install -r …` |

### REGLA PARA TU ENTORNO

No instales paquetes con `pip` dentro de `env-ia-mlops` salvo que se te pida. Si lo haces, `verificar_s1.py` lo avisa («con otra version») y te da la línea que lo arregla: `conda install --force-reinstall paquete=versión`. `conda env update` no revierte lo que cambió pip.

### Y EL PROYECTO BASE

Para S1 trabajas con el entorno del curso, sin instalar nada más: `04-Proyecto-Base` se ejecuta desde su carpeta, **sin** `pip install`. Las pruebas pasan así (comprobado en un entorno recién creado).

PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# Crea y activa el entorno env-ia-mlops

En: **Anaconda Prompt, en la carpeta del repositorio**

`environment.yml` es la receta: Python 3.11 y 23 paquetes con versión fijada, solo de conda-forge. Descarga cientos de MB: lánzalo con buena conexión y no lo interrumpas.

TERMINAL

```
> cd C:\cursos\curso-ml-ia-desarrollo-produccion
> conda env create -f 03-Entorno\conda\environment.yml
> conda activate env-ia-mlops
```

SI EL ENTORNO YA EXISTE: REPONER LO QUE FALTE O APLICAR CAMBIOS DEL ARCHIVO

```
> conda env update -f 03-Entorno\conda\environment.yml --prune
```

REHACERLO DESDE CERO

```
> conda deactivate
> conda env remove -n env-ia-mlops -y
> conda env create -f 03-Entorno\conda\environment.yml
```

PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# Crea y activa el entorno env-ia-mlops

## DEBE SALIR

Al final conda imprime cómo activarlo y el prompt cambia a **(env-ia-mlops)**. Si el prompt no lo muestra, el entorno NO está activo.

```
Preparing transaction:  ...working...  done
Verifying transaction:  ...working...  done
Executing transaction:  ...working...  done
#
# To activate this environment, use
#
#     $ conda activate env-ia-mlops
#
# To deactivate an active environment, use
#
#     $ conda deactivate

WARNING conda.conda_pypi.main:notify_conda_pypi_tip(158):
  Did you know? You can install many PyPI packages with conda
  using conda-pypi. Get started:
    https://bit.ly/4xXYt0B
```

Salida real en Linux (final del comando). Tardó 66 s con buena conexión; en Windows y con tu red puede tardar bastante más.

## SI FALLA

- `CondaToSNonInteractiveError`: haz antes el bloque B.
- `CondaValueError: prefix already exists`: el entorno ya existe; usa «Si el entorno ya existe» o «Rehacerlo desde cero» (más arriba en este paso).
- Se corta a mitad por la conexión o el disco: repite el comando; `conda clean --all` libera espacio.
- En PowerShell, `conda activate` necesita `conda init powershell` y una terminal nueva; es más fácil usar Anaconda Prompt.

Creación ejecutada en Linux desde environment.yml el 09/11/2026. En Windows solo se comprobó que resuelve (--dry-run).

PROBADO EN LINUX

# Comprueba que Python es el del entorno

En: **Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio**

La prueba de humo importa las librerías del curso y recorre un mini ciclo completo: capas en SQLite, entrenamiento con MLflow y una API. No necesita Internet.

TERMINAL

```
> python --version
> python -c "import sys; print(sys.executable)"
> python 03-Entorno\conda\prueba_de_humo.py
```

 PROBADO EN LINUX

# Comprueba que Python es el del entorno

## DEBE SALIR

Python 3.11.x, una ruta que contiene `env-ia-mlops` y, en la **última línea**, **HUMO OK**.

```
Python 3.11.18
```

```
<conda>/envs/env-ia-mlops/bin/python
```

```
python 3.11.18
  ok    pandas                2.3.3
  ok    pyarrow               26.0.1
…
  ok    papermill             2.7.0
  ok    jupytext              1.19.6
  ok    nbclient              0.11.0
  sqlite: tablas ['fs_cliente', 'master_ventas', 'ml_dataset', 'raw_ventas']
  mlflow: ejecucion b2bda262 | auc 0.701
  api: respuesta {'clase': 0}
  api: entrada invalida rechazada (422)
HUMO OK
```

Salida real en Linux (se acortó el centro). En Windows verás tus rutas.

## SI FALLA

- Si la ruta NO contiene el nombre del entorno, estás usando otro Python: activa el entorno y repite.
- Si sale **FALLA** junto a una librería, el entorno quedó incompleto: rehazlo (paso D1, «Rehacerlo desde cero»). Copia el mensaje completo para el instructor.

 PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# VS Code: mismo Python en scripts y cuadernos

En: VS Code

El error más común es dejar el kernel equivocado en un cuaderno. Si el intérprete o el kernel no son los del entorno, los cuadernos no encuentran las librerías.

1. En la terminal con el entorno activo, abre la carpeta: `code .`
2. Ctrl+Shift+P → **Python: Select Interpreter** → elige **env-ia-mlops**.
3. Abre `03-Entorno/conda/prueba_notebook.ipynb` → arriba a la derecha **Seleccionar kernel** → **Python Environments** → **env-ia-mlops**.
4. Pulsa **Ejecutar todo**. La primera celda muestra una ruta con `env-ia-mlops`; la última imprime **NOTEBOOK OK**.

TERMINAL

```
> python -m ipykernel install --user --name env-ia-mlops --display-name env-ia-mlops
```

solo si el kernel no aparece en la lista; reinicia VS Code después

Cópialas una a una: algunas van en otra terminal o llevan tus datos.

## DEBE SALIR

Cada celda se ejecuta sin errores y la última imprime **NOTEBOOK OK**.

```
Installed kernelspec env-ia-mlops in ~/.local/share/jupyter/kernels/env-ia-mlops
```

Salida real del registro del kernel en Linux (tu ruta será otra).

## SI FALLA

- La primera celda falla con «El kernel NO es env-ia-mlops»: elige el kernel correcto (paso 3).
- No aparece el entorno en la lista: ejecuta el comando de este paso y reinicia VS Code.

El cuaderno se ejecutó con `jupyter nbconvert` en Linux (NOTEBOOK OK) y el comando del kernel se probó; los clics en VS Code no se probaron.

PROBADO EN LINUX Y EN WINDOWS

# Pruebas del proyecto base

En: **Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio**

`04-Proyecto-Base` es el proyecto con el que se trabaja en S1. Sus pruebas confirman que el entorno hace lo que el curso espera. No hace falta instalar el paquete con `pip` para S1.

TERMINAL

```
> cd 04-Proyecto-Base
```

```
> python -m pytest -q -m "not lento"
```

```
> python -m pytest -q
```

opcional: incluye ejecutar los 5 cuadernos (más lento)

Cópialas una a una: algunas van en otra terminal o llevan tus datos.

PROBADO EN LINUX Y EN WINDOWS

# Pruebas del proyecto base

## DEBE SALIR

Rápidas: **127 passed, 4 deselected**. Completas: **131 passed**. Cualquier `failed` o `error` hay que contarlo al instructor.

```
.....................................................................  [ 56%]
.....................................................                  [100%]
127 passed, 4 deselected in 83.71s (0:01:23)
```

```
.....................................................................  [ 54%]
.....................................................                  [100%]
131 passed in 134.24s (0:02:14)
```

Salida real en Windows (11/11/2026). Los segundos dependen de tu equipo.

## SI FALLA

- Solo ves puntos y no la línea «127 passed»: tu copia es anterior al 13/11/2026. Actualiza con `git pull`.
- Sale **30**, **34**, **40** o **108 passed**: tu copia es anterior al 11/11/2026 (antes de juntar la base operacional, los pipelines y la API de S2). Actualiza con `git pull`.
- `ModuleNotFoundError`: entorno equivocado. Mira que el prompt empiece por (env-ia-mlops).
- Si falla alguna prueba, copia la salida completa y envíala; no la edites.

Ejecutado en Linux en un entorno recién creado desde environment.yml (09/11/2026) y en Windows 11 con la base operacional y los pipelines ya incluidos (11/11/2026), sin instalar nada más; recapturado en el Codespace del curso (13/11/2026).

PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# SQLite

En: **Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio**

Todas las capas del curso (raw, master, fs, ml) viven en una base SQLite. Python ya trae SQLite; solo hay que comprobar la versión y que ejecuta consultas.

TERMINAL

```
> python -c "import sqlite3; print(sqlite3.sqlite_version,
sqlite3.connect(':memory:').execute('SELECT 1 AS
entorno_listo').fetchone())"
```

OPCIONAL: UN VISOR PARA ABRIR LAS BASES (DB BROWSER FOR SQLITE)

```
> winget install --id DBBrowserForSQLite.DBBrowserForSQLite -e --accept-
source-agreements --accept-package-agreements
```

Ábrelo, crea una base nueva y, en «Ejecutar SQL», ejecuta `SELECT 1 AS entorno_listo;` : una fila con el valor 1.

Cópialas una a una.

## DEBE SALIR

Una versión de SQLite (3.25 o superior) y `(1,)`.

```
3.54.1 (1,)
```

Salida real en Linux; tu versión de SQLite puede ser otra.

## SI FALLA

- `ModuleNotFoundError` o un error raro: el Python no es el del entorno. Repite la comprobación del paso D2.

El comando se ejecutó en Linux; el visor DB Browser, no.

PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# MLflow: abre la interfaz

En: **Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio**

MLflow guarda parámetros, métricas y modelos de cada entrenamiento. En S1 lo usarás con una base SQLite. Aquí solo compruebas que arranca y se ve en el navegador.

TERMINAL

```
> cd C:\cursos
```

fuera del repositorio: crea un archivo mlflow.db que no queremos en el curso

```
> mlflow ui --backend-store-uri sqlite:///mlflow.db
```

EN S1 (DESDE 04-PROYECTO-BASE, DESPUÉS DE EJECUTAR EL CUADERNO 04)

```
> mlflow ui --backend-store-uri sqlite:///models/mlflow.db
```

1. Abre `http://127.0.0.1:5000` en el navegador: debe verse la pantalla de **Experiments**.
2. Para parar el servidor: **Ctrl+C** en esa terminal. Borra después `C:\cursos\mlflow.db`.

## DEBE SALIR

La terminal se queda ocupada (es normal) mostrando `Uvicorn running on http://127.0.0.1:5000`. Tarda unos segundos en estar listo.

```
INFO:     Uvicorn running on http://127.0.0.1:5000
(Press CTRL+C to quit)
```

Salida real en Linux: tardó 21 s en responder.

## SI FALLA

- El puerto 5000 está ocupado: usa `mlflow ui --port 5001 --backend-store-uri sqlite:///mlflow.db` y abre el puerto 5001.
- El navegador no abre la página: espera 20 s y refresca; comprueba que escribiste `127.0.0.1`, no otra dirección.

El servidor se arrancó y respondió en Linux; el navegador en Windows, no.

 PARTE PROBADA EN LINUX · EL RESTO SIN PROBAR EN WINDOWS

# El dato del caso: Bank Marketing

En: **Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio**

El caso que vas a desarrollar en el curso usa este archivo. Es de solo lectura: léelo con Python, no lo abras y guardes con Excel (cambiaría su contenido).

TERMINAL

```
> python -c "import pandas as pd; d = pd.read_csv('02-Proyecto-Transversal/datos/bank-marketing/bank-additional-full.csv', sep=';'); print(d.shape); print(d['y'].value_counts().to_dict())"
```

OPCIONAL: COMPROBAR LA HUELLA A MANO (POWERSHELL)

```
> Get-FileHash -Algorithm SHA256 02-Proyecto-Transversal\datos\bank-marketing\bank-additional-full.csv
```

Debe coincidir con la línea de `02-Proyecto-Transversal/datos/SHA256SUMS`. El verificador del paso siguiente lo hace por ti.

Cópialas una a una.

## DEBE SALIR

`(41188, 21)`: 41 188 filas y 21 columnas (el separador es `;`), y el reparto de la variable objetivo `y`.

```
(41188, 21)
{'no': 36548, 'yes': 4640}
```

Salida real en Linux.

## SI FALLA

- Otras dimensiones: el archivo está modificado. Recupéralo con `git restore 02-Proyecto-Transversal/datos/bank-marketing/bank-additional-full.csv`.
- `FileNotFoundError`: no estás en la carpeta del repositorio, o clonaste `main` en lugar de `develop`.

El comando de Python se ejecutó en Linux; `Get-FileHash` es de Windows y no se probó.

 PROBADO EN LINUX

# La comprobación final: verificar_s1.py

En: **Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio**

Reúne las comprobaciones anteriores y otras (versiones frente a environment.yml, Git, disco, puerto 5000, huella del dato) y dice qué hacer en cada fallo. Si lo ejecutas con `--informe`, guarda `verificacion-s1.txt` para enviarlo al instructor.

TERMINAL

```
> cd C:\cursos\curso-ml-ia-desarrollo-produccion
> python 03-Entorno\conda\verificar_s1.py --con-pruebas --informe
```

PROBADO EN LINUX

# La comprobación final: verificar_s1.py

## DEBE SALIR

Todas las líneas en **OK** y la última: **LISTO PARA S1**. **AVISO** = revísalo, no bloquea. **FALLA** = arréglalo antes de S1 (la propia línea dice cómo).

```
Verificacion para S1 - 2026-11-13 07:42 - Linux 6.8.0-1070-azure - Python 3.11.18
Entorno: env-ia-mlops   Repositorio: curso-ml-ia-desarrollo-produccion

  [OK   ] python                    3.11.18
  [OK   ] entorno                   env-ia-mlops
  [OK   ] paquetes                  23 paquetes con la version fijada en environment.yml
  [OK   ] importaciones             12 librerias clave se importan (3.1 s)
  [OK   ] sqlite                    version 3.54.1; consultas con funciones de ventana
correctas
  [OK   ] mlflow                    version 3.18.0; guarda y lee una ejecucion en SQLite
  [OK   ] kernel                    kernel «env-ia-mlops» registrado
  [OK   ] git                       git version 2.47.3
  [OK   ] git: identidad            nombre y correo configurados (no se muestran)
  [OK   ] repositorio               material del curso completo en la carpeta clonada
  [OK   ] dato Bank Marketing       huella correcta; 41188 filas de datos (+ encabezado)
  [OK   ] disco                     42 GB libres
  [OK   ] puerto 5000               libre para «mlflow ui»
  [OK   ] pruebas del proyecto base  127 passed, 4 deselected in 82.81s (0:01:22) (86 s)

Resultado: LISTO PARA S1  (14 OK, 0 AVISO, 0 FALLA)

Informe guardado en verificacion-s1.txt (en la carpeta actual). Enviaselo al instructor si
algo no se resuelve.
```

Salida real en Linux (la línea de pruebas, recapturada en Windows el 11/11/2026). El informe no incluye tu nombre de usuario ni tu correo de Git.

## SI FALLA

- Sale **FALLA** en «python» y «entorno»: no tienes el entorno activo. Empieza por ahí: casi todo lo demás es consecuencia.
- Sale **AVISO** en «paquetes»: algún paquete tiene otra versión (suele ser un `pip install` posterior). La línea «que hacer» trae el comando exacto: `conda install --force-reinstall paquete=versión`. `conda env update` no lo arregla (comprobado).

Ejecutado en el entorno recién creado, con la identidad de Git puesta como en A1: termina en LISTO PARA S1. El verificador tiene pruebas propias en 03-Entorno/conda/pruebas.

 SIN PROBAR EN WINDOWS

# Google Cloud y Codespaces (no bloquean S1)

En: **CMD o PowerShell**

Hacen falta desde S2 y S3. En S0 solo te autenticas y eliges tu proyecto: **no crees recursos de pago**. Docker y Kubernetes no se instalan en tu equipo: se usan dentro de Codespaces.

TERMINAL

```
> winget install --id Google.CloudSDK -e --accept-source-agreements --accept-package-agreements
```

```
> gcloud --version
```
en una terminal NUEVA

```
> gcloud auth login
```

```
> gcloud config set project TU_ID_DE_PROYECTO
```
el ID del proyecto, no su nombre

```
> gcloud auth list
```

Cópialas una a una: algunas van en otra terminal o llevan tus datos.

### DEBE SALIR

Tu cuenta aparece con un asterisco (activa) en `gcloud auth list`.

### SI FALLA

- Si no puedes instalar gcloud, usa Cloud Shell desde el navegador: no necesita instalación ni autenticación local.
- Docker en Codespaces: sigue las diapositivas 23 y 25 de la presentación principal de S0.

# Errores frecuentes

| LO QUE VES | CAUSA | QUÉ HACER |
|---|---|---|
| `conda` no se reconoce | Terminal equivocada. | Usa «Anaconda Prompt (Miniconda3)» desde el menú Inicio. |
| `CondaToSNonInteractiveError` | Términos de servicio de Anaconda sin aceptar. | Haz el bloque B (solo conda-forge) y repite la creación. |
| `CondaValueError: prefix already exists` | El entorno ya existe. | `conda env update -f 03-Entorno\conda\environment.yml --prune`, o bórralo y créalo. |
| `ModuleNotFoundError` | Entorno o kernel equivocado. | El prompt debe empezar por (env-ia-mlops). En un cuaderno, elige ese kernel. |
| Faltan carpetas del curso | Clonaste `main`. | `git fetch origin` y `git switch develop`. |
| pytest solo muestra puntos | Copia anterior al 13/11/2026. | `git pull`. Debe verse «127 passed». |
| «la huella no coincide» | El CSV de Bank Marketing se modificó. | `git restore` de ese archivo; no lo guardes con Excel. |
| `mlflow ui` no abre | Puerto 5000 ocupado o servidor aún arrancando. | `--port 5001`; espera 20 s y refresca. |

SI ALGO FALLA (2 DE 2)

# Si algo falla y cómo pedir ayuda

### QUÉ ENVIAR AL INSTRUCTOR

- **El comando** que ejecutaste y **en qué terminal** (CMD, PowerShell, Anaconda Prompt, VS Code).
- **El mensaje de error completo**, copiado como texto (no recortado).
- El archivo `verificacion-s1.txt` (lo crea `verificar_s1.py --informe`; no lleva tu usuario ni tu correo).

### QUÉ NO ENVIAR NUNCA

- Contraseñas, tokens ni claves de ningún servicio.
- Capturas de pantalla con tu sesión de GitHub o de Google Cloud abierta.

### SI TU EQUIPO NO TE DEJA INSTALAR NADA

Usa la **ruta sin instalación**: Codespaces y Cloud Shell desde el navegador. Está en la guía interactiva `03-Entorno/guia-instalacion/index.html` (sección «Ruta sin instalación: Codespaces») y en `03-Entorno/guias/docker-y-kubernetes-sin-instalacion-local.md`. Avisa antes de S1.

FINAL

# Listo para S1 cuando...

- [ ] `git --version` responde y `git branch --show-current` dice **develop** — A1 · C1
- [ ] `code --list-extensions` muestra las 7 extensiones — A2
- [ ] `conda --version` responde en Anaconda Prompt — A3
- [ ] `conda activate env-ia-mlops` y la prueba de humo termina en **HUMO OK** — D1 · D2
- [ ] El cuaderno de prueba imprime **NOTEBOOK OK** con el kernel env-ia-mlops — E1
- [ ] `pytest` del proyecto base: **127 passed** (y 131 con las lentas) — E2
- [ ] SQLite, MLflow UI y el dato de Bank Marketing responden — E3 · E4 · E5
- [ ] `verificar_s1.py` termina en **LISTO PARA S1** — E6

0 de 15 pasos marcados como hechos en este navegador

En S1 empezamos con `conda activate env-ia-mlops` y `cd 04-Proyecto-Base`.

TRANSPARENCIA

# Qué se probó y qué no

### PROBADO 09/11/2026 (LINUX)

- Creación del entorno desde cero con `environment.yml`, y **HUMO OK**.
- Versiones instaladas = versiones fijadas (verificador).
- Pruebas del proyecto base (hoy 127 rápidas y 131 completas, recapturadas en el Codespace del curso el 13/11/2026).
- SQLite, MLflow UI, lectura del CSV de Bank Marketing, registro del kernel y ejecución del cuaderno de prueba.
- `git clone --branch develop` contra GitHub (clon superficial).

### SIN PROBAR

- Nada de esto se ejecutó en un Windows real: ni `winget`, ni «Anaconda Prompt», ni `code`, ni los clics en VS Code o en el navegador.
- El clonado se probó contra GitHub (clon superficial, Linux): trae la rama `develop` con las carpetas del curso. No se probó en Windows ni el clon completo de principio a fin con una conexión lenta.
- Los tiempos (creación del entorno, MLflow UI) son de Linux con buena conexión; en clase serán mayores.
- El aviso de términos de servicio de conda solo lo da el Miniconda de Anaconda; el conda usado aquí viene de conda-forge y no lo reproduce.

Si algo de lo «sin probar» falla en tu equipo, es información valiosa: cuéntalo con el comando y el mensaje completo.